



Amrita School of AI
COIMBATORE CAMPUS

RELEASED WEEK 12 · DUE WEEK 15

Assignment 4: The Accelerator, and What It Is For

Port a kernel to CUDA, profile it properly, and connect what you find to how large models are actually trained and served.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

RELEASED

Week 12

DUE

End of week 15

WEIGHT

1.5% of the course (assignments total 6%)

MARKS

30

WORK

Individual

OFFERING

Odd Semester 2026-27

The task

Port one kernel to CUDA, optimise it with profiler evidence, and then answer a short analytical section connecting your measurements to Unit 4.

Suggested kernels: matrix transpose, 2-D convolution, sparse matrix-vector product, histogram, prefix sum, N-body force accumulation, or your Assignment 2 kernel.

Part A: Correct, then measured (8 marks)

A1. Working CUDA implementation, verified against a CPU reference at several sizes including sizes that are not multiples of your block size.

A2. Error checking on every CUDA call, and a demonstration that it catches something: deliberately break the launch configuration and show the error and where it was reported.

A3. Timing breakdown: host-to-device, kernel, device-to-host. Report the percentages.

A4. Compare against a good OpenMP version on the node's CPUs, both for a single call and for data that stays resident across many calls. State the crossover.

Part B: Optimise with evidence (12 marks)

At least three optimisations, each measured on its own, each justified by a profiler observation rather than a guess. Candidates: coalescing, shared-memory tiling, bank-conflict padding, occupancy tuning, reducing divergence, more work per thread, streams and overlap.

For each: the profiler evidence that motivated it, the prediction, the measurement, and whether the prediction held.

B1. An Nsight Systems timeline before and after, showing the GPU utilisation change.

B2. Nsight Compute metrics for your dominant kernel before and after: SM throughput, DRAM throughput, achieved occupancy, L2 hit rate, registers per thread.

B3. A measured roofline placement, from `ncu --set roofline`. State the fraction of attainable reached before and after.

B4. Compare your final kernel against the relevant vendor library (cuBLAS, cuDNN, cuFFT, CUB or Thrust) if one exists. Report the ratio honestly.

PITFALL · A TEXTBOOK SPEEDUP IS NOT A MEASUREMENT

You will find published claims about your optimisation. Reproduce them on the hardware you were allocated and report what you actually got. Where the numbers disagree with the literature, explain the disagreement using your own profiler data. That analysis is worth more marks than a large speedup would be.

Part C: Connect it to Unit 4 (10 marks)

Short analytical answers, using your own measured machine numbers where relevant.

C1. For a transformer of X billion parameters in FP16, compute the memory required for mixed-precision training with Adam. Show the per-parameter breakdown. Does it fit on one GPU of the node you used? If not, name the technique that addresses it and explain what it trades away.

C2. For the same model, compute the gradient size and the bytes each GPU sends per ring all-reduce step. Using a plausible interconnect bandwidth, estimate the communication time per step. Compare with a plausible computation time and state whether the network dominates.

C3. Compute the arithmetic intensity of single-token generation for that model. Place it relative to your GPU's ridge point. Is inference memory bound or compute bound?

C4. Using C3, explain in roofline terms why batching improves inference throughput so much, and name two other techniques that move the kernel in the same direction.

C5. One paragraph: which of the optimisations you performed in Part B has a direct analogue in large-model training or inference? Name it and explain the correspondence.

What to submit

`assign4-<ROLLN0>.pdf` plus source. Include both timelines, the metric tables, the measured roofline, and your Part C working.

MARKS	FOR
8	Part A: correctness, error handling demonstrated, transfer breakdown, CPU comparison
12	Part B: three optimisations, each motivated by profiler evidence and measured
10	Part C: correct arithmetic and a genuine connection to your own measurements

ON ASAICOMPUTE · GETTING A GPU

All GPU work goes through the scheduler. Build with `-arch=sm_80` so your binary runs on whichever node you are allocated. `ncu` may need `--set basic` if performance counters are restricted; if so, say so in your report rather than omitting the section.